

Parallel Programming for Computer Graphics

Mid-term project: 2D Circle Rasterization

Francesco Acciaioli

University of Florence

francesco.acciaioli@stud.unifi.it

Abstract

Rasterizing large numbers of alpha-blended, depth-ordered 2D primitives is a core building block of real-time and offline rendering, and it is embarrassingly parallel in principle but easy to implement inefficiently. This report presents a CPU-only 2D circle rasterizer, implemented in Python with Numba, that draws N alpha-blended circles onto an $H \times W$ image using the painter's algorithm. Four kernels of increasing sophistication are compared: a sequential baseline, a naive per-pixel parallel kernel, a tile-based parallel kernel with bounding-box culling, and a tile-based kernel additionally written for auto-vectorization (branchless SIMD inner loop). On an 8-core Apple Silicon (M2) machine, the tiled, vectorized kernel reaches a maximum speedup of $9.1\times$ over the sequential baseline (on 16 oversubscribed threads), while a dedicated scalar-vs-SIMD comparison shows vectorization alone contributes roughly $4\times$ ($3.6\text{--}5.5\times$) at a fixed thread count. A broader set of micro-benchmarks additionally isolates the effect of resolution, circle count, tile size, memory layout (SoA vs. AoS) and weak scaling, providing a systematic picture of where the performance comes from and where it saturates.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

This report presents the design, implementation and performance analysis of a sequential and parallelized 2D circle rasterizer. The project was developed for the *Parallel Programming* course to demonstrate how coarse-grained multithreading and fine-grained vectorization can be combined to accelerate a classic, embarrassingly-parallel computer-graphics workload.

1.1. Problem Description and Motivation

Rasterization is the process of converting geometric primitives into pixels. This project targets a deliberately simple but representative primitive set: N circles, each defined by a 2D center, a depth coordinate z , a radius, an RGB color and an alpha (opacity) value. Circles are composited onto the image using the *painter's algorithm*: they are sorted by increasing depth and drawn back-to-front, blending each new circle onto the frame buffer with the standard “over” operator

$$\text{out} = \text{src}_{\text{rgb}} \cdot a + \text{dst}_{\text{rgb}} \cdot (1 - a).$$

This is the same compositing model used by real rendering pipelines (e.g. particle systems, 2D vector graphics, UI compositors), which makes the problem a realistic, minimal stand-in for a much larger class of rendering workloads.

1.2. Computational Challenges

A pixel (i, j) belongs to circle k if its center $(i + 0.5, j + 0.5)$ satisfies $dx^2 + dy^2 \leq r_k^2$. In the naive formulation every one of the $H \times W$ pixels must be tested against every one of the N circles, giving $O(H \cdot W \cdot N)$ work per frame. Even after restricting each circle to its axis-aligned bounding box, the workload remains large and highly *irregular*: circles vary enormously in radius (a few pixels up to $\sim 12\%$ of the image side), so the amount of work associated with different pixels — and with different spatial tiles of the image — is far from uniform. This irregularity, rather than raw arithmetic volume, is the central challenge that the parallelization strategy has to address.

1.3. Objectives and Scope

The goal of the project is to explore, in a controlled way, which parallelization and vectorization techniques are effective for this class of workload, and why. The scope includes:

1. **Sequential implementation:** a straightforward, single-threaded kernel used as the correctness and performance baseline.
2. **Parallel optimization:** two coarse-grained multi-threaded strategies (per-pixel-row and per-tile), plus a tile-based kernel whose inner loop is written to be auto-vectorized by the compiler.
3. **Systematic micro-benchmarking:** isolating, one variable at a time, the effect of thread count, image resolution, circle count, tile size, SIMD, memory layout and weak scaling.

2. Algorithm Description

2.1. Data Structure

Circles are stored in a *Structure-of-Arrays* (SoA) container (`CircleSoA`): eight parallel, contiguous `float32` NumPy arrays $(x, y, z, \text{radius}, r, g, b, a)$, rather than a list of per-circle objects. This layout is cache-friendly and is the natural starting point for vectorization, since each attribute is unit-stride in memory [3]. Sorting by depth is done once, up front, with a *stable* mergesort, so that circles with equal z keep their original (insertion) order deterministically; the sort is never repeated inside a rendering kernel.

2.2. Sequential Algorithm

The baseline kernel loops over circles from far to near. For each circle it computes an integer bounding box clipped to the image, then scans only the pixels inside that box, testing the circle equation and applying the “over” blend in place:

```
for k in range(n): # far -> near
    x0,y0,x1,y1 = bbox(k) # clipped to image
    for j in range(y0, y1):
        for i in range(x0, x1):
            if inside_circle(i, j, k):
                blend_over(img[j, i], color(k))
```

This bounding-box restriction is the only optimization present in the sequential version; it is still $O(N)$ circles times, on average, $O(r_k^2)$ pixels, and it writes directly into a shared image buffer, which is safe here only because the loop is single-threaded.

2.3. Parallelization Strategy

Two independent opportunities for parallelism were identified:

Parallelizing across the image (multithreading). Because the final color of a pixel does not depend on the color of any other pixel (circles are looked up, not accumulated across pixels), the image can be partitioned into disjoint regions — rows or square tiles — and each region processed independently by a different thread, with no shared mutable state and therefore no locks.

Vectorizing the inner loop (SIMD). Inside a tile, testing a circle against many pixels performs the same arithmetic (subtraction, multiplication, comparison, blend) on independent data. If that inner loop is written without early-exit branches and with a small, contiguous, per-tile buffer, the compiler’s auto-vectorizer can turn it into SIMD instructions that process several pixels per cycle.

3. Parallel Implementation

Four rendering kernels are implemented and compiled just-in-time with Numba, which generates native code through LLVM (@njit); they are referred to as `seq`, `pixels`, `tiles` and `simd`, and all four are required to produce the same image (up to floating-point reassociation error from `fastmath`).

3.1. Strategy Overview

Mode	Parallelism	Notes
<code>seq</code>	none	baseline, bbox scan
<code>pixels</code>	<code>prange</code> over rows	no culling, slow
<code>tiles</code>	<code>prange</code> over tiles	bbox culling per tile
<code>simd</code>	tiles + vector inner loop	fastest, branchless

Table 1: The four rendering kernels compared in this project.

`pixels` parallelizes Numba’s `prange` over image rows: each thread owns a disjoint set of rows and, for every pixel, loops over *all* N circles with a cheap bounding-box rejection test. It has no spatial culling structure, so it is included mainly as a “multithreading alone” reference point — it is intentionally slow.

`tiles` instead parallelizes over square tiles of the image. For each tile, a thread-local candidate list is built once by testing which circles’ bounding boxes intersect the tile; only that (usually much smaller) candidate set is then scanned against every pixel of the tile. Disjoint tiles mean disjoint writes, so again no synchronization is needed.

`simd` uses the same tile/culling structure as `tiles`, but the inner double loop over a tile’s pixels is restructured for auto-vectorization: for each candidate circle, all pixels of the tile are visited in a countable, branch-free loop that computes a 0/1 mask instead of using an `if`, and pixels are accumulated into small contiguous per-tile R/G/B scratch buffers before being written back to the image once at the end:

```
for ii in range(tw): # vectorizable
    d2 = dx*dx + dy2 # FMA under fastmath
    m = 1.0 if d2 <= r2 else 0.0 # branchless mask
    rr[idx] = m*(psr + rr[idx]*inv) + (1-m)*rr[idx]
```

Removing the branch and keeping the accumulator arrays contiguous and small (tile-sized) lets LLVM auto-vectorize the loop with unit-stride vector loads/stores, and lets `fastmath=True` fuse the distance computation into fused multiply-adds.

3.2. Load Balancing: prange and Scheduling

Numba’s `prange` is the direct analogue of an OpenMP `#pragma omp parallel for`: it distributes loop iterations (rows, or tiles) across the threads set by `numba.set_num_threads()`. Because circle radii vary widely, the amount of work per tile is highly non-uniform (a tile with only small, sparse circles finishes almost instantly; a tile covered by several large overlapping circles is far more expensive). This is exactly the kind of irregular workload for which *dynamic* scheduling is preferable to static partitioning. Unlike OpenMP, however, `prange` has no `schedule(static|dynamic|guided)` clause, so the three strategies cannot be compared directly. By default Numba splits the iteration space into roughly equal contiguous blocks, one per thread, which behaves like `schedule(static)`. The load is balanced instead by splitting the image into many small tiles (many more tiles than threads): each thread’s block then contains many tiles, so cheap and expensive tiles average out. This is why the tile size is studied explicitly in Section 5.

3.3. Synchronization

By construction, every kernel here operates on disjoint output regions (rows or tiles), so the renderer itself is **lock-free**: no shared mutable accumulator is ever written by more than one thread, and no locks, atomics or critical sections are required.

3.4. Data Structures and Memory Layout

The default layout is SoA, as described above. For comparison, a `to_aos()` conversion produces an interleaved ($N, 8$) *Array-of-Structures* layout, used only by the memory-layout benchmark (Section 5.5); the renderer itself never uses it internally.

4. Experimental Setup

4.1. Environment

Benchmarks were executed on an Apple Silicon MacBook Air M2 (ARM64), a hybrid design with 4 performance and 4 efficiency cores (8 logical/physical cores reported by the OS). The software stack was Python 3.11.9, NumPy 2.4.6, Numba 0.67.0 and llvmlite 0.49.0 on macOS. Numba’s thread ceiling (`NUMBA_NUM_THREADS`) was deliberately set to **16**, i.e. *twice* the number of logical cores, so that the thread-scaling experiment can observe oversubscription behavior (thread count exceeding the physical core count) as required by the course guidelines [1], in addition to normal scaling up to the core count. The MacBook Air is fanless, so under sustained all-core load it reduces its clock frequency (thermal throttling): single-threaded timings were reproducible across runs, while multi-threaded timings varied by up to $\sim 20\%$ between runs. Multi-threaded results should therefore be read as trends rather than exact values.

4.2. Dataset

The dataset is procedurally generated: N circles with uniformly random position, radius (scaled to image resolution), color and alpha. This is a deliberate choice rather than a limitation — for a 2D rasterizer, synthetically generated geometry *is* the realistic workload, in the same way standard rasterization benchmarks use synthetic scenes rather than an external image dataset; there is no natural pre-existing dataset of “circles with depth and alpha” to substitute.

4.3. Methodology and Parameters

- **Timing:** wall-clock time via `time.perf_counter` equivalents, with a per-configuration time budget that stops adding iterations once exceeded (bounding the cost of slow configurations), plus CPU time (`time.process_time()`, summed across threads) and an approximate 95% confidence interval ($1.96 \cdot \sigma / \sqrt{n}$).
- **Warm-up:** each configuration is preceded by 2 warm-up runs (excluded from timing) to absorb Numba’s JIT compilation cost; kernels are also cached to disk (`@jit(cache=True)`) across runs.
- **Iterations:** 5 timed iterations per configuration (subject to the time budget).
- **Scaling dataset:** the strong-scaling experiment (§5.1) deliberately uses a large problem (2048^2 pixels, 100,000 circles) so that the sequential baseline takes ≥ 10 s, as



Figure 1: Example output of the rasterizer: several thousand alpha-blended, depth-sorted circles composited with the “over” operator.

required by the course’s benchmarking guidelines [1] for meaningful speedup measurements.

- **Validation:** before any timing, the benchmark script renders the same scene with every kernel (`pixels`, `tiles`, `simd` and the `AoS` variant) at the maximum thread count and compares each image with the sequential one. The check uses a tolerance of 10^{-3} on the maximum absolute pixel difference rather than bit-exact equality, since `fastmath=True` permits floating-point reassociation, and it stops the run if any kernel exceeds it. Two cases are tested: a 512^2 image and a 1000×700 image, whose sides are not multiples of the tile size (partial tiles at the borders). The largest difference measured was $1.2 \cdot 10^{-7}$, i.e. float32 rounding.

5. Results and Analysis

5.1. Strong Scaling

Figure 2 shows time, speedup and parallel efficiency versus thread count for a fixed, large problem (2048^2 pixels, 100,000 circles; sequential baseline: 13.55 s). Table 2 reports the same data for the `simd` kernel.

Three effects are visible. First, vectorization alone already gives the `simd` kernel a large head start at $p=1$ ($2.5\times$ faster than `seq` on a single thread), confirming that SIMD and multithreading are largely independent, additive sources of speedup. Second, all three kernels scale well up to 4–8 threads, matching the machine’s physical core count. Third, beyond 8 threads (i.e. once the thread count exceeds the number of physical cores, entering oversubscription on the efficiency cores) speedup flattens for every

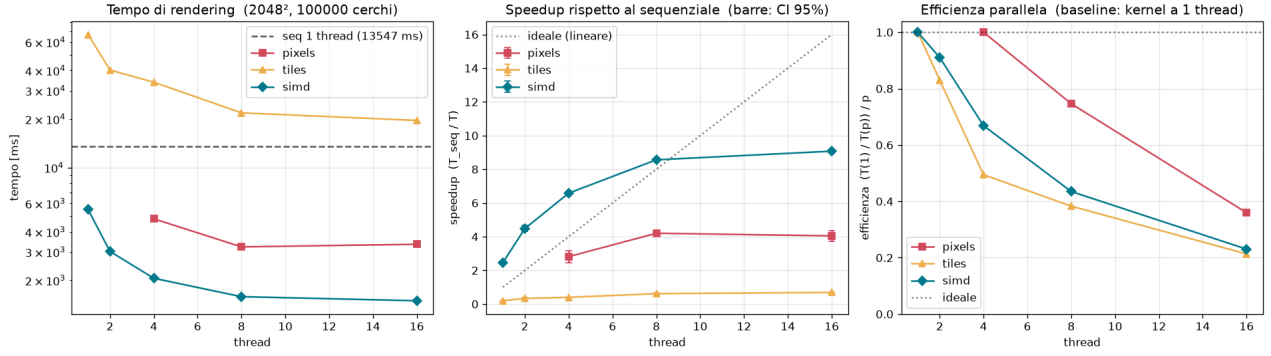


Figure 2: Strong scaling: rendering time (log scale), speedup vs. the sequential baseline, and parallel efficiency, for all three parallel kernels, from 1 to 16 (oversubscribed) threads.

Threads	Time (ms)	Speedup	Efficiency
1	5503.5	2.46×	2.46
2	3022.8	4.48×	2.24
4	2058.3	6.58×	1.65
8	1582.2	8.56×	1.07
16	1492.9	9.08×	0.57

Table 2: Strong scaling of the `simd` kernel on the 100,000-circle, 2048^2 workload (speedup relative to the 13.55 s sequential baseline; efficiency here is speedup/ p).

kernel while efficiency keeps dropping — the extra logical threads provide diminishing, then negligible, returns. The `tiles` kernel (culling, no SIMD) is consistently between `pixels` and `simd`, showing that culling and vectorization are complementary, not substitutable, optimizations: culling alone (`tiles`) only reaches $\sim 0.7\times$ the sequential kernel’s speedup at best, while adding SIMD on top pushes the maximum speedup to **9.08×**. On this very dense workload (100,000 large circles) each tile has a long candidate list, so `tiles` tests many circles per pixel, whereas `seq` only visits the pixels inside each circle’s bounding box.

5.2. Effect of Resolution and Circle Count

Figures 3–4 sweep image resolution and circle count independently (at a fixed thread count of 16). Both scale roughly linearly with problem size for all three kernels tested (`seq`, `tiles`, `simd`), consistent with the algorithm’s cost being dominated by pixels-inside-bounding-boxes rather than by a resolution- or count-dependent constant overhead. `simd` is always the fastest, 2–5× faster than `tiles`, with the gap growing with problem size. `tiles` beats `seq` in most configurations (at 2048^2 , `tiles` is $2.3\times$ faster than `seq`; `simd` is $8.5\times$ faster), but falls behind it at 8000 circles (235.8 ms vs. 196.0 ms): with many circles the per-tile candidate lists grow long, the same effect seen in the strong-scaling experiment.

5.3. Effect of Tile Size

Figure 5 sweeps the tile side length from 16 to 256 pixels (1024^2 image, 4000 circles, 16 threads). `simd` is fastest at

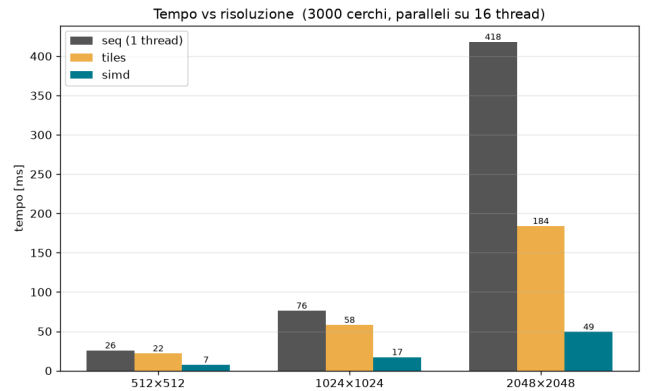


Figure 3: Rendering time vs. image resolution (3000 circles, 16 threads).

tile = 32 and `tiles` at tile = 16 (with 32 only 10% slower); both degrade quickly for larger tiles: bigger tiles mean fewer, coarser parallel work units (worse load balancing and less L1/L2 residency for the per-tile scratch buffers), while very small tiles pay a proportionally larger fixed per-tile overhead (building the candidate list). `simd` is consistently 3–4.3× faster than `tiles` across the whole sweep, and both kernels prefer small tiles (16–32 px), so the tile-size sweet spot depends little on whether the inner loop is vectorized.

5.4. Effect of SIMD (Scalar vs. Vectorized)

To isolate the contribution of vectorization from thread count, Figure 6 compares `tiles` (scalar inner loop) against `simd` (vectorized inner loop) across the same thread sweep (1–16), at fixed resolution and circle count. At every thread count `simd` is faster: **3.6×** at $p=1$ (327.4 ms vs. 91.2 ms) and still **4.0×** at $p=16$ (91.1 ms vs. 22.9 ms). The peak of $5.5\times$ at $p=4$ is partly due to `tiles` scaling poorly from 2 to 4 threads in this run (184.9 ms $\rightarrow$ 144.1 ms), rather than to `simd` itself. This confirms that vectorization and multithreading are two genuinely separate levers on performance, and that neither substitutes for the other.

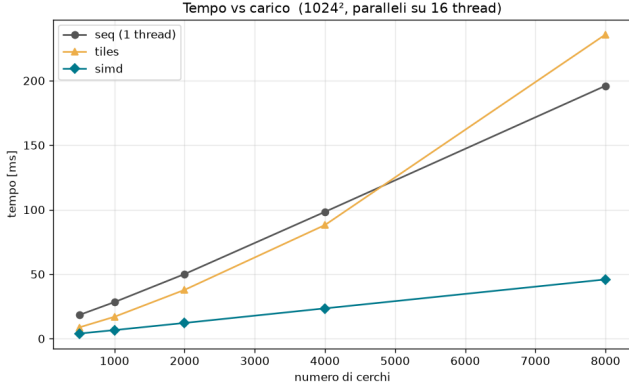


Figure 4: Rendering time vs. circle count (1024² image, 16 threads).

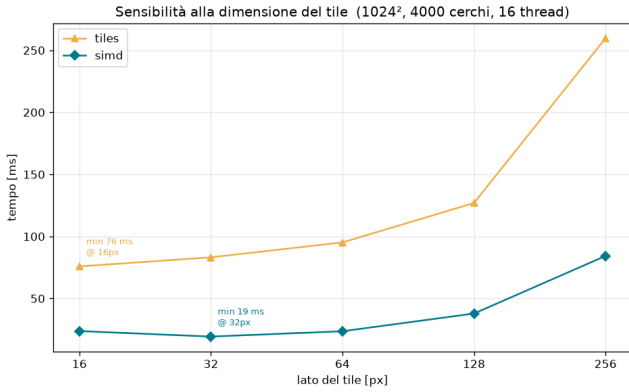


Figure 5: Rendering time vs. tile size, for the `tiles` and `simd` kernels.

5.5. Memory Layout: SoA vs. AoS

Figure 7 compares the default SoA layout against an interleaved AoS layout for the otherwise-identical `tiles` kernel. The `tiles` kernel was chosen over `simd` on purpose: in `tiles`, circle attributes are read inside the per-pixel loop (for every candidate circle at every pixel), so it is the kernel most sensitive to how circles are laid out in memory. In `simd`, instead, each circle's attributes are loaded once per tile, before the vectorized loop, which then only touches the per-tile R/G/B buffers; since SIMD there works across pixels, not across circles, the circle layout would have almost no effect. Neither layout wins **consistently**: AoS is faster at 512² (24.5 vs. 28.9 ms) and 2048² (188.5 vs. 209.6 ms), SoA at 1024² (63.9 vs. 67.7 ms). Differences of 6–15% that change sign from one resolution to the next point to measurement variability rather than to a real layout effect. This is expected for this workload: circle attributes are read once per tile-candidate-test and once per pixel hit, not streamed sequentially over the whole array in a tight loop, so the unit-stride advantage of SoA is largely hidden by the small candidate-list sizes and by cache residency of the per-tile working set — the memory-layout choice matters far less here than it would for a bulk, streaming numerical kernel.

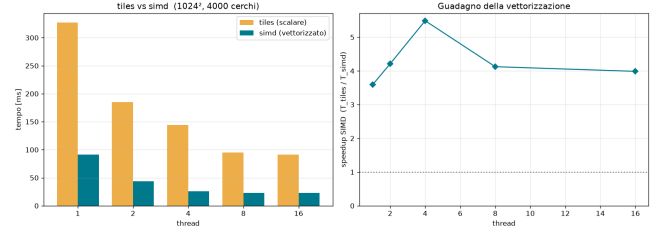


Figure 6: Scalar (`tiles`) vs. vectorized (`simd`) kernel, across thread counts 1–16 (1024² image, 4000 circles).

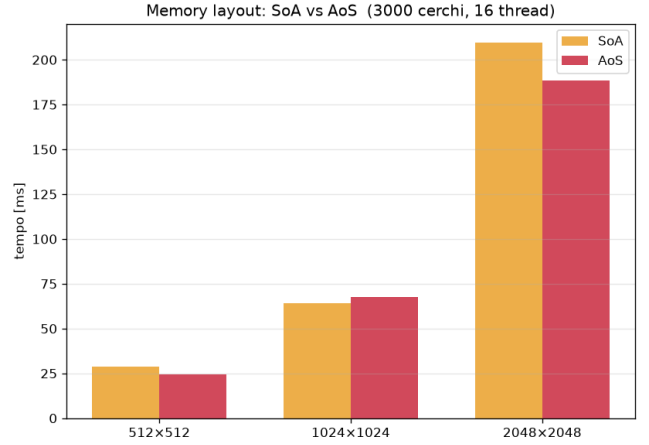


Figure 7: Rendering time, SoA vs. AoS circle storage, across resolutions.

5.6. Weak Scaling

Figure 8 keeps work-per-thread constant by growing the circle count proportionally to the thread count (500 circles per thread). Efficiency is ≥ 1.0 up to 4 threads (i.e. within the physical core count, where larger batches amortize fixed per-call overhead better than the 1-thread case), then drops to 0.65 at 8 threads and 0.34 at 16. The two drops have different causes. At 8 threads there is no oversubscription yet (8 threads on 8 cores), but threads 5–8 run on the slower efficiency cores, so the threads on the performance cores finish early and wait for them. At 16 threads the machine is oversubscribed (16 threads on 8 cores): the extra threads add no compute and efficiency halves again. Both effects match the strong scaling result, so they are properties of the machine rather than of a specific fixed problem size.

5.7. Comparison with Amdahl's and Gustafson's Laws

Amdahl's law [2] bounds the speedup on p threads as $S(p) = 1/(f + (1 - f)/p)$, where f is the fraction of the run that cannot be parallelized. In the `simd` wrapper the only serial work is allocating the image and sorting the circles by Z : about 17 ms out of the 5.5 s single-thread run on the strong scaling workload, so $f \approx 0.003$. With this f , Amdahl predicts $S(8) \approx 7.8$ and $S(16) \approx 15.3$ relative to `simd` on one thread. The measured values (Table 2) are much lower: $5503.5/1582.2 = 3.5$ at 8 threads and 3.7 at 16. The Karp–

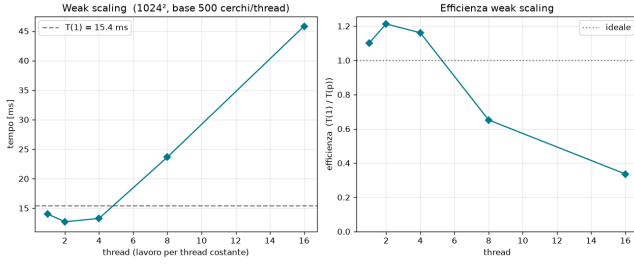


Figure 8: Weak scaling: circle count grows proportionally with thread count (500 circles/thread); efficiency relative to the 1-thread baseline.

Flatt metric $e = (1/S(p) - 1/p)/(1 - 1/p)$, which estimates the serial fraction from the measured speedup, gives $e = 0.10, 0.17, 0.19$ and 0.22 for $p = 2, 4, 8, 16$. It is far above 0.003 and grows with p . A growing e indicates that the limit is not serial code but an overhead that increases with the thread count: the slower efficiency cores from 5 threads on, thermal throttling under all-core load, and oversubscription at 16 threads.

Gustafson’s law [3] describes the weak scaling case, where the problem grows with p : the scaled speedup is $S(p) = p - f(p - 1)$, so with $f \approx 0.003$ the efficiency should stay close to 1. This matches the measurements up to 4 threads (efficiency ≥ 1.0 , Fig. 8) but not beyond (0.65 at 8, 0.34 at 16), confirming the same conclusion: past the performance cores, scaling is limited by the hardware, not by the serial part of the algorithm.

5.8. Bottleneck Analysis

Taken together, the experiments point to two dominant, largely independent limits on performance:

- **Heterogeneous cores and oversubscription.** Every scaling experiment (strong and weak) shows the same qualitative behavior: good scaling up to 4 threads (the performance cores), smaller gains from 5 to 8 threads, when the slower efficiency cores join in, and no further gain beyond 8 threads, where the extra threads only oversubscribe the 8 physical cores and add scheduling overhead.
- **Load imbalance from circle-size variance,** which is exactly why tile-based `prange` scheduling and tile-size tuning matter more than memory-layout choices for this workload: unlike a dense numerical kernel, the bottleneck here is uneven *work* distribution across parallel units, not memory bandwidth or cache locality (as confirmed by the lack of a consistent SoA/AoS difference).

6. Conclusions

This project implemented and systematically benchmarked a CPU-parallel 2D circle rasterizer, combining coarse-grained multithreading (Numba `prange` over image tiles) with fine-grained SIMD vectorization (a branchless, auto-vectorizable

inner loop). The tiled, vectorized kernel reaches a maximum speedup of **9.08×** over the sequential baseline on an 8-core Apple Silicon M2, with vectorization alone contributing roughly **4×** independently of thread count. Culling (tile-based candidate lists) and vectorization proved to be complementary, additive optimizations rather than substitutes for one another. A dedicated tile-size sweep identified small tiles (32 px for `simd`) as the sweet spot for this workload, and, since tiles are disjoint, the renderer needs no synchronization at all. Memory layout (SoA vs. AoS) had a comparatively minor effect, consistent with a workload whose bottleneck is irregular per-tile work rather than memory bandwidth. Both strong and weak scaling confirm that scaling is good on the 4 performance cores, weaker once the efficiency cores are involved, and flat under oversubscription beyond 8 threads.

Future work could explore spatial acceleration structures (e.g. a tile-local BVH or grid over circles for very high circle counts), GPU offload of the tile-based kernel, and adaptive tile sizing driven by local circle density rather than a single fixed tile size.

References

- [1] Parallel Programming course, Università degli Studi di Firenze, (course slides: assignment descriptions, implementation guidelines and technical report structure).
- [2] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proc. AFIPS Spring Joint Computer Conference*, 1967.
- [3] R. Robey, Y. Zamora, *Parallel and High Performance Computing*, Manning, 2021.

Authorship: I confirm that I am the sole author of this writeup and the associated code.